

NLP

classmate

Date _____

Page _____

Natural Language Processing

- NLP is part of AI that helps computer understand, read, write and talk like humans.
- Human talk in languages like Hindi, Eng, Gujarati But computers only understand no (0 & 1). To bridge this gap, we need NLP so that computers can read and understand.

Application :

- Chatbots & virtual Assistants
- Email spam filtering
- Sentiment Analysis

Approaches of NLP :

(I) Rule-Based Approach

- Early method of solving NLP using handwritten rules and grammatical logic.

ex: "not good" → negative sentiment

- if sentence contains 'not good', mark it as negative sentiment.

used for :

- Grammar checker
- spell checker
- simple key-word chatbots

Limitation :

- can't handle complex, ambiguous or new patterns
- Not scalable for large data

(2) Statistical (ML) approach

→ Language is treated like data, and algorithm learn pattern from it.

→ Model used: used for:

→ Naive Bayes → sentiment analysis

→ Logistic Reg. → spam detection

→ SVM → Text classification

→ Requires text to be converted into numbers using techniques like "Bag of words", "TF-IDF" etc.

Limitation:

→ Loses context of words (like sarcasm or sequence)

ex:

1) → "Yeah, b'coz staying up all night debugging is exactly what i dreamed of"

ML thinks: 'dreamed', 'exactly' → sounds like pos

Reality: It's painful dev life

2) I love it when the internet stops working during a meeting.

ML thinks: 'love', 'Meeting' → positive

Reality: Sarcastic rage

(3) Deep-Learning based Approach (Modern ML)

→ "Use neural networks to automatically learn complex patterns & context from text."

Popular Architecture:

- RNN (Recurrent Neural Network)
- LSTM (Long-Short Term Memory)
- GRU (Gated Recurrent Units)
- Transformers (BERT, GPT, T5, etc)

Pipeline

Step-1: Gathering the text or say getting data

- 1) You can use the pre build public data set for this
- 2) You can do web scraping
- 3) APIs (structured + legal way to get live data)
- 4) Manual collection / crowdsourcing
- 5) Use NLP tools to auto-generate text

Step-2: Text cleaning

- 1) Lowercasing
- 2) Remove punctuation (., ! ? # @ \$ % ^ & * () etc.
 → They don't add meaning in ML context
 → In DL based NLP punctuation can carry meaning, and we usually keep it during preprocessing.
- 3) Remove Numbers (depends on task)
- 4) Remove URLs/links
- 5) Remove HTML tags
- 6) Remove emojis & special characters
- 7) Remove stopwords

→ Common word like is, the, was, and, in etc.

→ These don't help in understanding the meaning.

- All other task is easy but for removing stop words we have to use NLTK (Natural Language Toolkit). also Spacy as well.

Corpus :

- A collection of texts.
ex: A folder of 10000 tweets is your corpus

- "Text is 1 msg, corpus is whole collection."

Sentence : A group of words that form a meaningful statement.

Token : Each individual words or symbol in text is called token.

token = ["I", "Love", "AK"]

- The process of splitting text into tokens called Tokenization.

Feature Extraction / vectorization

- one hot encoding
- Bag of words
- TF-IDF
- Word2Vec
- BERT / Transformers

- Text is not a numbers. But ML models only understand numbers not feelings, words, emojis.

- So we convert word → numbers called "Feature Extraction"

(1) One-Hot Encodings

Document: Single piece of text. it can be one sentence, or paragraph or even full article depends on project.

Vocabulary: The list of all unique words present in your corpus.

EX:

I love pizza & I love pasta
 ↳ ["I", "Love", "pizza", "pasta"]

EX!

- D1) Amit watch movies
 D2) Ajay also watch movie.
 D3) Movie teach Amit.

Amit	watch	movies	Ajay	also	teach
------	-------	--------	------	------	-------

(D1) $D1 = \begin{bmatrix} [1, 0, 0, 0, 0, 0] \\ [0, 1, 0, 0, 0, 0] \\ [0, 0, 1, 0, 0, 0] \end{bmatrix}$ (3,6)

$D2 = \begin{bmatrix} [0, 0, 0, 1, 0, 0] \\ [0, 0, 0, 0, 1, 0] \\ [0, 1, 0, 0, 0, 0] \\ [0, 0, 1, 0, 0, 0] \end{bmatrix}$ (4,6)

pros:

- simple & easy to implement
- works well for small vocabularies

cons:

- out of vocabularies: vocabulary is limited to the words in your initial input, model cannot handle any new words.

→ Semantic Meaning: it fails to capture the relationship context, or meaning b/w words.

→ size difference

② Bag of words

→ convert text-to-numbers: It creates a list of all the unique words in your datasets, and then for each sentence, it counts how many times each word appears.

1) Amit watch shergins

2) Jeel also watch "

3) shergins teach shergins

} This ex uses multiple times in NLP, so remember

	Amit	watch	shergins	Jeel	also	teach
1	1	1	1	0	0	0
2	0	1	1	1	1	0
3	0	0	2	0	0	1

→ max_features: max count of feature

⇒ Sklearn, feature_extraction, text, CountVectorizer

Pros:

Cons

→ easy to implement

→ Sparse Matrix

→ fixed size

→ Out of vocabulary

→ work with ML

→ Semantic Meaning (Better one)

→ fast & efficient

→ Out of order

n -grams: n consecutive items from given text.
These items can be words, text, character etc.

Unigram ($n=1$): 'I', 'am', 'learning', 'NLP'.

Bigram ($n=2$): 'I am', 'am learning', 'learning NLP'.

Trigram ($n=3$): 'I am learning', 'am learning NLP'.

Pros:

Cons:

- Semantic meaning → Dimension increase
- little bit → OOV

③ TF-IDF (Term Frequency - Inverse Document Freq).

→ A statistical measure that evaluates the importance of word in a document relative to collection of doc.

→ TF: Measure how often a word appears in doc.

$$TF = \frac{\text{No of occurrence of term in doc}}{\text{Total no of term in doc}}$$

ex:

$$\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$$

range

$$\rightarrow (0, 1)$$

→ IDF: Measure how unique or informative term is across the corpus.

→ words that appears in many doc (and, is, the) are given lower weight, while rare term receive higher weight.

$$IDF = \log_e \left(\frac{\text{Total no of doc}}{\text{no of doc containing the term}} \right)$$

Ex: BoW

Amit $\rightarrow \log_e \frac{(3)}{1}$, Shreyans $\rightarrow \log_e \left(\frac{3}{3}\right)$
 $\downarrow$ sure $\rightarrow 0$

classmate

Date _____
Page _____

$$\boxed{TF-IDF = TF \times IDF}$$

Ex:

$$\text{Watch} \rightarrow \frac{1}{3} \times \log_e \left(\frac{3}{2}\right)$$

Pros:

Cons:

$\rightarrow$ Information $\rightarrow$ Sparsity
retrieval $\rightarrow$ poor
(search engine) $\rightarrow$ dimensions.